Mathematical Guide

All Models, Losses, Strategies, and Uncertainty Methods
in the AutoML Package

AutoML Research Project

April 2026

Contents

I Framework **4**

1 Introduction and Notation **4**
1.1 Notation 4

2 Uncertainty Quantification: A Unified View **4**
2.1 Taxonomy of Uncertainty Sources 5
2.2 Learned Variance via NLL 5
2.3 Law of Total Variance 5
2.4 MC Dropout 6
2.5 Binned Residual Standard Deviation 6
2.6 Summary: How Each Model Produces Uncertainty 6

II Currently Supported Models **7**

3 Linear Regression **7**

4 Gradient-Boosted Trees (XGBoost, LightGBM, CatBoost) **7**
4.1 Regularized Objective 7
4.2 Custom NLL Objective for Probabilistic Output 7
4.3 Implementation Differences 8

5 Plain PyTorch Neural Network **8**

6 Classifier-Regression Model **8**
6.1 Architecture Overview 8
6.2 Binning 8
6.3 Stage 1: Classification 9
6.4 Stage 2: Mapping 9
6.4.1 Linear Mapper 9
6.4.2 Lookup Mapper (`LOOKUP_MEDIAN` / `LOOKUP_MEAN`) 9
6.4.3 Spline Mapper 9
6.4.4 Neural Network Mapper 9

7 Probabilistic Regression Model (ProbReg) 9
7.1 Architecture Overview . . . 9
7.2 Class Masking for Variable K . . . 10
7.3 Regression Strategies . . . 10
7.3.1 Strategy 1: Separate Heads (`SEPARATE_HEADS`) . . . 10
7.3.2 Strategy 2: Single Head, N Outputs (`SINGLE_HEAD_N_OUTPUTS`) . . . 11
7.3.3 Strategy 3: Single Head, Final Output (`SINGLE_HEAD_FINAL_OUTPUT`) . . . 11
7.4 Law of Total Variance Aggregation . . . 11
7.5 Gradient Stop Variant . . . 11
7.6 Monotonic Constraints . . . 11
7.7 Boundary Constraints . . . 12

8 Flexible Neural Network (FlexibleNN) 12
8.1 Architecture . . . 12
8.2 Soft Forward Pass . . . 12
8.3 Hard Forward Pass (Inference) . . . 13
8.4 Independent-Weights Variant . . . 13

III Dynamic Architecture Selection 14

9 Selection Strategies 14
9.1 None Strategy (Fixed) . . . 14
9.2 Soft-Gating . . . 14
9.3 Gumbel-Softmax . . . 14
9.4 Straight-Through Estimator (STE) . . . 14
9.5 REINFORCE . . . 15
9.6 Strategy Summary . . . 15

10 Regularization for Dynamic Selection 15
10.1 Penalty . . . 15
10.2 ELBO with KL Divergence . . . 15
10.3 Unified View: ELBO as Occam's Razor . . . 16
10.4 Direct Regression Channel (ProbReg only) . . . 16

IV Loss Functions and Training 17

11 Loss Functions 17
11.1 Gaussian Negative Log-Likelihood (NLL) . . . 17
11.2 β-NLL Loss (Seitzer et al., 2022) . . . 17
11.3 Masked Cross-Entropy Loss . . . 17
11.4 Combined Training Loss (ProbReg) . . . 17
11.5 Boundary Regularization Loss . . . 17
11.6 Middle-Class Penalty . . . 17
11.7 Tree-Model NLL Objective . . . 18

12 Learned Regularization 18
12.1 L1 Only (Laplace Prior) . . . 18
12.2 L2 Only (Gaussian Prior) . . . 18
12.3 L1 + L2 (Horseshoe-like Prior) . . . 18

13 Target Transforms **18**
13.1 Symmetric Log Transform (Symlog) . . . 18
13.2 Uncertainty Conversion: Monte Carlo . . . 18

14 Training Loop **18**

V Proposed Baseline Models **20**

15 NGBoost (Duan et al., 2020) **20**

16 Mixture Density Network — MDN (Bishop, 1994) **20**

17 Deep Ensembles (Lakshminarayanan et al., 2017) **20**

18 Quantile Regression NN (Koenker, 2005) **20**

19 Gaussian Process **21**

20 Constant-Variance NN **21**

21 FT-Transformer (Gorishniy et al., 2021) **21**

22 TabPFN (Hollmann et al., 2024) **21**

23 Laplace-Approximation BNN (Daxberger et al., 2021) **21**

24 Conformal Prediction **22**

VI Evaluation **23**

25 Evaluation Metrics **23**
25.1 Currently Implemented . . . 23
25.2 Proposed Additions . . . 23
25.2.1 CRPS . . . 23
25.2.2 Winkler Score . . . 23
25.2.3 PIT-Based Calibration (Kuleshov et al., 2018) . . . 23
25.2.4 Sharpness . . . 23
25.2.5 Pinball Loss . . . 23
25.2.6 MPIW . . . 24

26 Photometric Redshift Metrics **24**

A Known Bugs Affecting the Mathematics **24**

Part I

Framework

1 Introduction and Notation

This document provides a complete mathematical specification of every model, loss function, selection strategy, uncertainty method, and evaluation metric in the AutoML package — both currently implemented and proposed.

1.1 Notation

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^d$	Input feature vector (d features)
$y \in \mathbb{R}$	Scalar regression target
N	Number of training samples
K	Number of classes / bins / mixture components
D	Number of hidden layers (depth) in FlexibleNN
M	Number of ensemble members or MC samples
$\pi_k(\mathbf{x})$	Probability of class k given input $\mathbf{x}$
$\mu_k(\mathbf{x})$	Mean prediction of regression head k
$\sigma_k^2(\mathbf{x})$	Variance prediction of regression head k
$\hat{y}(\mathbf{x})$	Final point prediction
$\hat{\sigma}^2(\mathbf{x})$	Final variance prediction
$\boldsymbol{\theta}$	All learnable parameters
$\boldsymbol{\phi}$	Parameters of a selection / gating subnetwork
τ	Temperature parameter (Gumbel-Softmax)
α	Miscoverage level for prediction intervals
β	Variance weighting exponent (β-NLL)
$k(\mathbf{x}, \mathbf{x}')$	Kernel / covariance function (GP)
η	Learning rate

Tensor shapes are written in parentheses: (B, K) means batch size B, K columns. We write $[n]$ for $\{1, 2, \ldots, n\}$.

2 Uncertainty Quantification: A Unified View

Every model in this package produces (or can produce) an uncertainty estimate $\hat{\sigma}^2(\mathbf{x})$. This section provides the taxonomy *before* describing individual models, so the reader has the mental model up front.

2.1 Taxonomy of Uncertainty Sources

Method	How $\hat{\sigma}^2$ is produced	What it captures	Input-dep?
Learned variance (NLL)	Second output of NN, trained via NLL	Aleatoric	Yes
Law of Total Variance	Aggregation of per-class (μ_k, σ_k^2)	Aleatoric + epistemic-like	Yes
MC Dropout	Empirical variance of M stochastic passes	Epistemic (approx.)	Yes
Deep Ensembles	Variance across M independent models	Aleatoric + epistemic	Yes
Binned Residual Std	Variance of training residuals in bins	Aleatoric (discretized)	Partially
Lookup Mapper	Per-probability-bin variance	Aleatoric (discretized)	Partially
GP Posterior	Analytical posterior variance	Aleatoric + epistemic	Yes
Conformal	Calibration-set residual quantile	Coverage guarantee	No (marginal)
Constant	Global scalar, not learned	None (placeholder)	No

2.2 Learned Variance via NLL

The network outputs $(\mu(\mathbf{x}), \log \sigma^2(\mathbf{x}))$ and is trained with the Gaussian NLL (§11.1). The gradient w.r.t. $\log \sigma^2$:

$$\frac{\partial \mathcal{L}}{\partial \log \sigma^2} = \frac{1}{2}\left(1 - \frac{(y-\mu)^2}{\sigma^2}\right),$$

drives σ^2 toward $\mathbb{E}[(y-\mu)^2 \mid \mathbf{x}]$, the true conditional variance, at convergence.

Used by: ProbReg (per-head), Plain NN (`PROBABILISTIC`), NN Mapper, Deep Ensembles (per member), NGBoost, tree models (custom objective).

2.3 Law of Total Variance

Given per-class predictions $\{(\mu_k, \sigma_k^2)\}_{k=1}^K$ weighted by $\{\pi_k\}_{k=1}^K$:

$$\hat{\sigma}^2 = \underbrace{\sum_k \pi_k \sigma_k^2}_{\text{expected within-class variance}} + \underbrace{\sum_k \pi_k (\mu_k - \hat{y})^2}_{\text{between-class variance}} . \tag{1}$$

The first term is aleatoric (noise within each class). The second term captures disagreement among class means. When all μ_k agree, the second term vanishes and we get pure aleatoric uncertainty.

Used by: ProbReg (central novelty), MDN, Deep Ensembles (with uniform $\pi_m = 1/M$).

2.4 MC Dropout

Gal & Ghahramani (2016): keep dropout active at test time and run M forward passes:

$$\hat{y}_m = f_{\boldsymbol{\theta},\mathbf{z}_m}(\mathbf{x}), \quad \mathbf{z}_m \sim \text{Bernoulli}(1-p)^L, \quad m = 1, \ldots, M, \tag{2}$$

$$\hat{\mu} = \frac{1}{M}\sum_{m=1}^{M} \hat{y}_m, \qquad \hat{\sigma}^2 = \frac{1}{M}\sum_{m=1}^{M} (\hat{y}_m - \hat{\mu})^2. \tag{3}$$

Used by: Plain NN (with `MC_DROPOUT`).

2.5 Binned Residual Standard Deviation

A post-hoc method. After training:

1. Compute residuals: $r_i = y_i - \hat{y}_i$ for all training samples.
2. Bin predictions into B bins by $\hat{y}_i$.
3. For each bin b: $\hat{\sigma}_b = \text{std}(\{r_i : \hat{y}_i \in b\})$.
4. At test time: $\hat{\sigma}(\mathbf{x}) = \hat{\sigma}_{b(\hat{y}(\mathbf{x}))}$.

Partially input-dependent: piecewise constant (one value per bin).
Used by: Linear Regression, XGBoost, LightGBM, CatBoost, Plain NN (with `BINNED_RESIDUAL_STD`).

2.6 Summary: How Each Model Produces Uncertainty

Model	**Primary UQ Method**	**Alternatives**
Linear Regression	Binned Residual Std	—
Plain NN	Depends on `uncertainty_method`	CONSTANT / PROBABILISTIC / MC_DROPOUT / B
XGBoost	Custom NLL objective (learned σ^2)	Binned Residual Std
LightGBM	Custom NLL objective (learned σ^2)	Binned Residual Std
CatBoost	Custom NLL objective (learned σ^2)	Binned Residual Std
ProbReg	Law of Total Variance	+ Conformal wrapper
ClassReg (Lookup)	Per-bin variance from lookup table	Conformal wrapper
ClassReg (Spline)	Constant residual variance	—
ClassReg (NN Mapper)	Learned σ^2 via NLL	—
FlexibleNN	Same as Plain NN	—
GP	Posterior variance (analytical)	—
Deep Ensembles	Ensemble disagreement + mean aleatoric	—
NGBoost	Natural gradient learned σ^2	—
MDN	Mixture variance (law of total variance)	—
Quantile Regression	Quantile-based intervals	—

Part II

Currently Supported Models

3 Linear Regression

Ordinary least squares:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad (\mathbf{w}, b) = \arg\min_{\mathbf{w},b} \sum_{i=1}^{N} (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2.$$

Uncertainty: No native uncertainty. Uses post-hoc `BINNED_RESIDUAL_STD` (§2.5).

4 Gradient-Boosted Trees (XGBoost, LightGBM, CatBoost)

All three share the same additive ensemble structure:

$$\hat{y}(\mathbf{x}) = \sum_{t=1}^{T} \eta \cdot f_t(\mathbf{x}), \quad f_t \in \mathcal{F},$$

where $\mathcal{F}$ is the space of decision trees, η is the learning rate (shrinkage), and each tree f_t is fit to the pseudo-residuals.

4.1 Regularized Objective

At step t:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{N} \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \gamma T_{\text{leaves}} + \frac{1}{2}\lambda \sum_{j=1}^{T_{\text{leaves}}} w_j^2, \tag{4}$$

where $g_i = \partial \mathcal{L} / \partial \hat{y}_i$ and $h_i = \partial^2 \mathcal{L} / \partial \hat{y}_i^2$.

4.2 Custom NLL Objective for Probabilistic Output

For trees predicting $(\mu, \log \sigma^2)$, custom gradients and Hessians:

Mean output:

$$g_\mu = \frac{\mu - y}{\sigma^2}, \qquad h_\mu = \frac{1}{\sigma^2}. \tag{5}$$

Log-variance output:

$$g_{\log \sigma^2} = \frac{1}{2}\left(1 - \frac{(y-\mu)^2}{\sigma^2}\right), \qquad h_{\log \sigma^2} = \frac{1}{2} \quad \text{(Fisher information)}. \tag{6}$$

Implementation Note

Bug N5: the current implementation uses the observed Hessian $h = \frac{1}{2}\frac{(y-\mu)^2}{\sigma^2}$ instead of the Fisher information $h = \frac{1}{2}$. The Fisher information is preferred because it is always positive and gives more stable tree splits.

4.3 Implementation Differences

- **XGBoost** (Chen & Guestrin, 2016): exact or approximate greedy split finding, L1/L2 regularization on leaf weights.
- **LightGBM** (Ke et al., 2017): histogram-based splits, gradient-based one-side sampling (GOSS), exclusive feature bundling (EFB).
- **CatBoost** (Prokhorenkova et al., 2018): ordered boosting to prevent target leakage, oblivious (symmetric) trees with balanced splits at each depth.

Uncertainty: Learned $\sigma^2(\mathbf{x})$ via custom NLL objective, or post-hoc `BINNED_RESIDUAL_STD`.

5 Plain PyTorch Neural Network

A feedforward MLP:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = W_L \cdot \sigma(W_{L-1} \cdots \sigma(W_1 \mathbf{x} + b_1) \cdots + b_{L-1}) + b_L,$$

where σ is an activation function (ReLU, GELU, etc.) and L is the number of layers.

Uncertainty modes:

- `CONSTANT`: No learned uncertainty. $\hat{\sigma}^2$ is a global constant.
- `PROBABILISTIC`: Output layer produces $(\mu, \log \sigma^2)$; trained with Gaussian NLL (§2.2).
- `MC_DROPOUT`: Dropout active at test time; run M forward passes (§2.4).
- `BINNED_RESIDUAL_STD`: Post-hoc binned variance (§2.5).

6 Classifier-Regression Model

A two-stage model: (1) discretize the target into bins and train a classifier, (2) map classifier probabilities back to regression predictions.

6.1 Architecture Overview

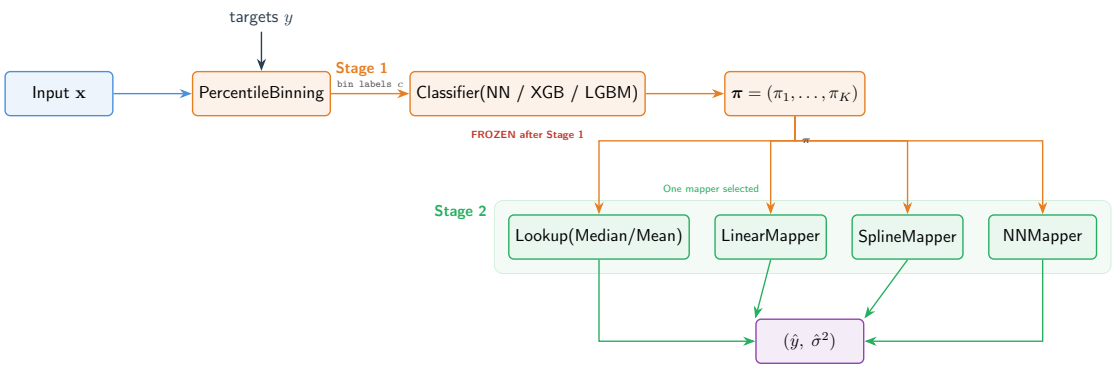

Figure 1: ClassifierRegression two-stage architecture. The classifier is frozen before Stage 2 (mapper) training.

6.2 Binning

Targets are binned using percentile-based boundaries:

$$q_j = \text{Percentile}\left(y_{\text{train}}, \frac{100 \cdot j}{K}\right), \quad j = 0, 1, \ldots, K, \tag{7}$$

$$c_i = \max\{j : y_i \geq q_j\}, \quad c_i \in \{0, 1, \ldots, K-1\}. \tag{8}$$

Duplicate boundaries (from repeated values) are collapsed, and K is reduced accordingly.

6.3 Stage 1: Classification

Train a classifier $f : \mathbb{R}^d \to \Delta^K$ (NN, XGBoost, LightGBM, or CatBoost) on $(\mathbf{x}_i, c_i)$ pairs. After training, **freeze** the classifier.

6.4 Stage 2: Mapping

A *mapper* transforms class probabilities to regression predictions.

6.4.1 Linear Mapper

$\hat{y} = a \cdot \hat{\pi} + b$, where $\hat{\pi}$ is a scalar summary of the probability vector and (a, b) are fit by OLS.

Uncertainty: Constant residual variance $\hat{\sigma}^2 = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$.

6.4.2 Lookup Mapper (`LOOKUP_MEDIAN` / `LOOKUP_MEAN`)

The most reliable mapper. Bins the probability space and computes per-bin statistics:

1. For each class k, collect $\pi_k(\mathbf{x}_i)$ across training samples.
2. Bin the probability values into B bins (where $B = \max(B_{\min}, \min(B_{\max}, N/100))$).
3. For each bin b:

$$\hat{y}_b = \text{median}(\{y_i : \pi_k(\mathbf{x}_i) \in b\}) \quad \text{(or mean)}, \tag{9}$$

$$\hat{\sigma}_b^2 = \text{var}(\{y_i : \pi_k(\mathbf{x}_i) \in b\}) \quad \text{(if PROBABILISTIC)}. \tag{10}$$

4. At prediction time: look up the bin for $\pi_k(\mathbf{x})$, return $(\hat{y}_b, \hat{\sigma}_b^2)$.

Uncertainty: Per-probability-bin variance — *input-dependent* through the probability: different π_k values land in different bins.

6.4.3 Spline Mapper

$\hat{y} = S(\pi_k; \text{degree} = k_s, \text{smoothing} = s)$ via `UnivariateSpline`. Known to be unstable.

Uncertainty: Constant residual variance (no per-input variance).

6.4.4 Neural Network Mapper

$(\hat{y}, \log \hat{\sigma}^2) = h_{\boldsymbol{\theta}(\boldsymbol{\pi})}$ where h is a small MLP. Trained with NLL loss (probabilistic) or MSE (deterministic).

Uncertainty: Learned input-dependent $\hat{\sigma}^2$ via NLL. Known training issues due to two-stage optimization.

7 Probabilistic Regression Model (ProbReg)

The central idea: discretize the prediction task into K classes via a learned classifier, attach a per-class regression head that outputs $(\mu_k, \log \sigma_k^2)$, and combine via the **law of total variance**.

7.1 Architecture Overview

1. **Classifier backbone** $f_{\boldsymbol{\theta}:\mathbb{R}^d \to \mathbb{R}^K}$. Produces raw logits $\ell_1, \ldots, \ell_K$. Softmax gives class probabilities:

$$\pi_k(\mathbf{x}) = \frac{\exp(\ell_k(\mathbf{x}))}{\sum_{j=1}^K \exp(\ell_j(\mathbf{x}))}.$$

2. **Regression module** (one of three strategies, §7.3).

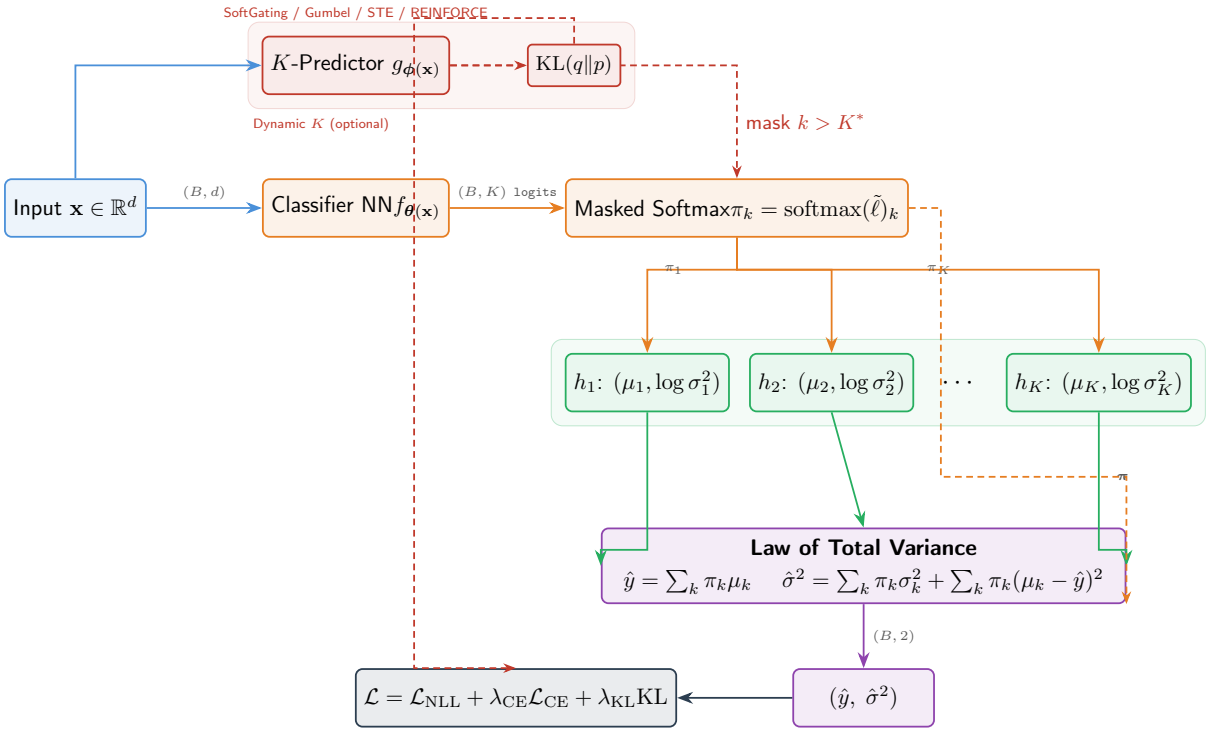

Figure 2: ProbReg architecture. Solid arrows: data flow. Dashed: optional dynamic-K path. Each regression head h_k receives only its own probability π_k (SEPARATE_HEADS strategy).

3. **Aggregation** via law of total variance (§2.3).
4. **Optional: dynamic** K via a gating subnetwork (§9).

7.2 Class Masking for Variable K

When using dynamic K or a fixed $K < K_{\max}$, classes $k > K$ are masked:

$$\tilde{\ell}_k = \begin{cases} \ell_k & \text{if } k \le K, \\ -\infty & \text{if } k > K. \end{cases}$$

7.3 Regression Strategies

Given class probabilities $\pi_1, \ldots, \pi_K$, the regression module maps these to per-class outputs $(\mu_k, \log \sigma_k^2)$ for $k \in [K]$.

7.3.1 Strategy 1: Separate Heads (`SEPARATE_HEADS`)

Each class k has an *independent* neural network head h_k:

$$h_k : \mathbb{R} \to \mathbb{R}^2, \quad h_k(\pi_k) = (\mu_k, \log \sigma_k^2).$$

The input to head k is the *scalar* probability $\pi_k(\mathbf{x})$. Heads do not share parameters.

Implementation Note

This is the primary strategy used in benchmarks. The key architectural choice: each head only sees its own class probability, not the full probability vector. The classifier acts as a nonlinear feature extractor projecting onto the probability simplex; each head learns a scalar-to-scalar function.

7.3.2 Strategy 2: Single Head, N Outputs (`SINGLE_HEAD_N_OUTPUTS`)

One shared neural network takes the full probability vector:

$$h : \mathbb{R}^K \to \mathbb{R}^{K\times 2}, \quad h(\boldsymbol{\pi}) = \{(\mu_k, \log \sigma_k^2)\}_{k=1}^K.$$

7.3.3 Strategy 3: Single Head, Final Output (`SINGLE_HEAD_FINAL_OUTPUT`)

One shared network maps probabilities to per-class representations:

$$h : \mathbb{R}^K \to \mathbb{R}^{K\times 2}, \quad \hat{y} = \sum_{k=1}^{K} \pi_k \cdot h_k(\boldsymbol{\pi}).$$

Remark 7.1. Strategies 2 and 3 allow parameter sharing across classes but lose the per-class specialization that makes `SEPARATE_HEADS` effective.

7.4 Law of Total Variance Aggregation

Definition 7.1 (Law of Total Variance Decomposition)**.**

$$\hat{y}(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{x}] = \sum_{k=1}^{K} \pi_k(\mathbf{x}) \cdot \mu_k(\mathbf{x}) \tag{11}$$

$$\hat{\sigma}^2(\mathbf{x}) = \mathrm{Var}(Y \mid \mathbf{x}) = \underbrace{\sum_{k=1}^{K} \pi_k(\mathbf{x}) \cdot \sigma_k^2(\mathbf{x})}_{\text{expected aleatoric variance}} + \underbrace{\sum_{k=1}^{K} \pi_k(\mathbf{x}) \cdot (\mu_k(\mathbf{x}) - \hat{y}(\mathbf{x}))^2}_{\text{variance of means (epistemic-like)}} \tag{12}$$

Implementation Note

In `pytorch_utils.apply_law_of_total_variance`: input shapes: probabilities (B, K), per-head outputs $(B, K, 2)$. Output: $(\hat{y}, \log \hat{\sigma}^2)$ with shape $(B, 2)$, where $\log \hat{\sigma}^2 = \log(\max(\hat{\sigma}^2, 10^{-9}))$.

7.5 Gradient Stop Variant

When `optimization_strategy = GRADIENT_STOP`: $\tilde{\pi}_k = \mathrm{stopgrad}(\pi_k)$. The classifier is updated only by cross-entropy; regression heads only by NLL.

7.6 Monotonic Constraints

Separate mean and variance heads with monotonic linear on the mean:

$$\mu_k = g_\mu(\pi_k; \mathbf{w}_\mu) \quad \text{(monotonic)}, \qquad \log \sigma_k^2 = g_\sigma(\pi_k; \mathbf{w}_\sigma) \quad \text{(unconstrained)}. \tag{13}$$

Monotonic linear:

$$f_{\text{mono}}(\mathbf{x}; \mathbf{w}, b) = \begin{cases} \mathrm{softplus}(\mathbf{w})^\top \mathbf{x} + b & \text{positive monotonicity}, \\ -\mathrm{softplus}(\mathbf{w})^\top \mathbf{x} + b & \text{negative monotonicity}, \end{cases}$$

where $\mathrm{softplus}(w) = \log(1 + e^w)$ ensures non-negative weights.

7.7 Boundary Constraints

`HARDSIGMOID` constrains predictions to $[y_{\min}, y_{\max}]$:

$$\tilde{\mu}_k = y_{\min} + (y_{\max} - y_{\min}) \cdot \text{hardsigmoid}(\mu_k).$$

Uncertainty: Law of total variance (§2.3) — the central novelty of ProbReg.

8 Flexible Neural Network (FlexibleNN)

A neural network with *learnable depth*: a gating subnetwork predicts how many hidden layers each input should use.

8.1 Architecture

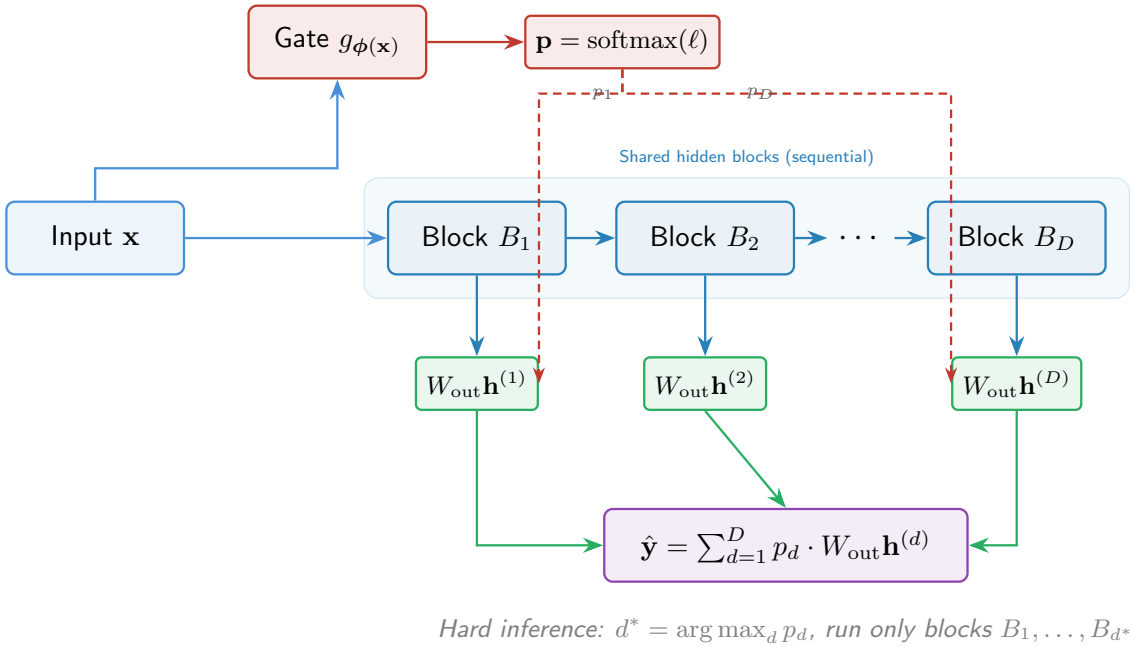

Figure 3: FlexibleNN architecture. Soft forward: all depth paths computed, weighted by p_d. Hard inference: only the argmax depth path is computed per sample.

- **Gate network** $g_{\phi:\mathbb{R}^d \to \mathbb{R}^{D_{\max}}}$.
- **Hidden blocks** $B_1, \ldots, B_{D_{\max}}$: each is Linear → BatchNorm → Activation → Dropout.
- **Output layer** $W_{\text{out}} : \mathbb{R}^h \to \mathbb{R}^o$, shared across all depths.

8.2 Soft Forward Pass

Depth probabilities: $\mathbf{p} = \text{softmax}(g_{\phi(\mathbf{x})})_{\in \Delta^{D_{\max}}}$.

For each depth $d \in [D_{\max}]$: $\mathbf{h}^{(d)} = B_d \circ \cdots \circ B_1(\mathbf{x})$.

Aggregated output:

$$\hat{\mathbf{y}} = \sum_{d=1}^{D_{\max}} p_d(\mathbf{x}) \cdot W_{\text{out}}\, \mathbf{h}^{(d)}. \tag{14}$$

Training cost is $O(D_{\max})$ per sample — all depth paths are computed.

8.3 Hard Forward Pass (Inference)

$d^*(\mathbf{x}) = \arg\max_d p_d(\mathbf{x})$. Group samples by depth, run only needed blocks:

Algorithm 1 Hard inference for FlexibleNN

```
for d = 1 to D_max do
    I_d ← {i : d*(x_i) = d}
    if I_d ≠ ∅ then
        ŷ_{I_d} ← W_out(B_d ∘ ··· ∘ B_1(x_{I_d}))
    end if
end for
```

Compute savings: $(D_{\max} - \bar{d})/D_{\max}$. Accuracy gap: $\sim$0.02 MSE.

8.4 Independent-Weights Variant

Each depth option has a *completely independent* network f_d with exactly d hidden layers (no shared blocks):

$$\hat{\mathbf{y}} = \sum_{d=1}^{D_{\max}} p_d(\mathbf{x}) \cdot f_d(\mathbf{x}).$$

This is a **mixture of experts**. More parameters ($O(h^2 \cdot D_{\max}^2/2)$ vs $O(h^2 \cdot D_{\max})$) but full per-depth specialization.

Uncertainty: Same as Plain NN (depends on `uncertainty_method`).

Part III

Dynamic Architecture Selection

The framework below is shared by ProbReg (selecting number of classes K) and FlexibleNN (selecting depth D). We use z as the generic discrete latent variable.

9 Selection Strategies

All strategies compute a probability vector $\mathbf{p} \in \Delta^{|\mathcal{Z}|}$ over possible values of z, then aggregate predictions.

9.1 None Strategy (Fixed)

$p_k = \delta_{k,k_{\text{fixed}}}$. No gating network.

9.2 Soft-Gating

Fully differentiable:

$$\mathbf{p} = \text{softmax}(g_{\boldsymbol{\phi}(\mathbf{x})}), \qquad \hat{\mathbf{y}} = \sum_k p_k \cdot \hat{\mathbf{y}}^{(k)}.$$

Implementation Note

SoftGating is the best-performing strategy for ProbReg dynamic-K. It achieves MSE matching the best fixed-K baseline with mean $K \approx 2$.

9.3 Gumbel-Softmax

Training: $p_k = \exp((\ell_k + G_k)/\tau) / \sum_j \exp((\ell_j + G_j)/\tau)$, $G_k \sim \text{Gumbel}(0, 1)$.
Evaluation: $p_k = \exp(\ell_k/\tau) / \sum_j \exp(\ell_j/\tau)$ (deterministic).
As $\tau \to 0$, samples approach one-hot. Aggregation same as Soft-Gating.

Remark 9.1. Gumbel-Softmax for dynamic K has poor empirical performance — the stochastic noise corrupts KL gradients in the ELBO formulation.

9.4 Straight-Through Estimator (STE)

Forward: $\mathbf{p}_{\text{hard}} = \text{one_hot}(\arg\max_k \ell_k)$.
Backward: Gradients flow through the soft distribution: $\mathbf{p}_{\text{STE}} = \mathbf{p}_{\text{hard}} - \text{stopgrad}(\mathbf{p}_{\text{soft}}) + \mathbf{p}_{\text{soft}}$.

Implementation Note

Bug N2: current STE implementation uses indexed assignment instead of the weighted-sum pattern, breaking the gradient path.

9.5 REINFORCE

Discrete sampling with policy gradient:

$$k^* \sim \text{Categorical}(\text{softmax}(g_{\boldsymbol{\phi}(\mathbf{x})})), \quad \hat{\mathbf{y}}=\hat{\mathbf{y}}^{(k^*)}.$$

Policy gradient loss:

$$\mathcal{L}_{\text{REINFORCE}} = -\log \pi_{k^*}(\mathbf{x}) \cdot R, \quad R = -\mathcal{L}_{\text{task}}^{\text{detach}}. \tag{15}$$

Remark 9.2. REINFORCE has high variance. Needs baseline subtraction $R \leftarrow R - \bar{R}$ (not yet implemented).

9.6 Strategy Summary

Strategy	Forward Pass	Gradient
None	$p_k = \delta_{k,k_{\text{fixed}}}$	N/A
Soft-Gating	$\mathbf{p} = \text{softmax}(\boldsymbol{\ell})$	Standard softmax gradient
Gumbel-Softmax	$p_k \propto e^{(\ell_k+G_k)/\tau}$	Reparameterization
STE	Forward: one_hot(arg max); backward: soft	Straight-through
REINFORCE	$k^* \sim \text{Cat}(\text{softmax}(\boldsymbol{\ell}))$	$-R \cdot \nabla \log \pi_{k^*}$

10 Regularization for Dynamic Selection

Without regularization, the gating network drifts to $z = z_{\max}$.

10.1 Penalty

Penalize expected complexity:

$$\mathcal{L}_{\text{penalty}} = \lambda \cdot \sum_k k \cdot p_k(\mathbf{x}). \tag{16}$$

Used as `K_PENALTY` (ProbReg) and `DEPTH_PENALTY` (FlexibleNN).

10.2 ELBO with KL Divergence

Treat z as a discrete latent variable with prior $p(z)$ favoring simplicity.

Prior:

$$p(z = k) \propto w_k, \quad (w_1, \ldots, w_{|\mathcal{Z}|}) = \text{linspace}(3.0, 1.0, |\mathcal{Z}|).$$

The `linspace(3, 1)` parameterization ensures constant steepness regardless of $|\mathcal{Z}|$.

Posterior: $q(z = k \mid \mathbf{x}) = p_k(\mathbf{x}) = \text{softmax}(g_{\boldsymbol{\phi}(\mathbf{x}))_k}$.

ELBO:

$$\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{task}} + \lambda_{\text{KL}} \cdot \text{KL}\big(q(z \mid \mathbf{x}) \,\|\, p(z)\big), \tag{17}$$

$$\text{KL}(q\|p) = \sum_k q(z = k \mid \mathbf{x}) \left[\log q(z = k \mid \mathbf{x}) - \log p(z = k)\right]. \tag{18}$$

Implementation Note

Bug N1: FlexibleNN uses `arange` instead of `linspace(3,1)`, causing prior steepness to scale with $D_{\max}$. ProbReg already has the fix.

10.3 Unified View: ELBO as Occam's Razor

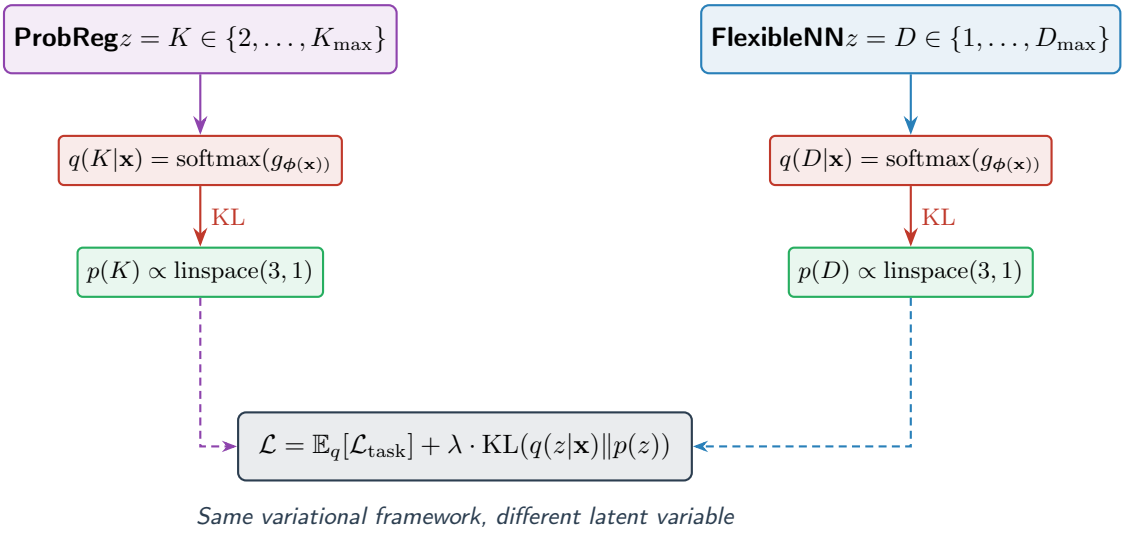

Figure 4: Unified ELBO framework. ProbReg selects K; FlexibleNN selects D. Both use the same KL penalty against a linspace prior.

This is the same principle as in variational autoencoders (Kingma & Welling, 2014), but applied to a *discrete architectural choice* rather than a continuous latent code.

10.4 Direct Regression Channel (ProbReg only)

When dynamic K is enabled, index 0 of the gating output corresponds to a *direct regression* bypass: $\hat{\mathbf{y}}^{(0)} = f_{\text{direct}}(\mathbf{x})$, allowing fallback to standard regression when the classification bottleneck adds no value.

Part IV

Loss Functions and Training

11 Loss Functions

11.1 Gaussian Negative Log-Likelihood (NLL)

$$\mathcal{L}_{\text{NLL}}(\hat{y}, \hat{\sigma}^2, y) = \frac{1}{2}\left(\log(2\pi) + \log\hat{\sigma}^2 + \frac{(y-\hat{y})^2}{\hat{\sigma}^2}\right). \tag{19}$$

Implementation Note

The $\log(2\pi)$ constant is included in `gaussian_nll_loss`. The β-NLL variant currently omits it (bug N4).

11.2 β-NLL Loss (Seitzer et al., 2022)

$$\mathcal{L}_{\beta\text{-NLL}} = \left(\hat{\sigma}^2_{\text{detach}}\right)^{\beta} \cdot \frac{1}{2}\left(\log\hat{\sigma}^2 + \frac{(y-\hat{y})^2}{\hat{\sigma}^2}\right). \tag{20}$$

$\beta = 0$: standard NLL. $\beta = 0.5$: recommended default. $\beta = 1$: can over-correct.
Intuition. Large $\hat{\sigma}^2$ amplifies the loss via $(\hat{\sigma}^2)^{\beta}$, discouraging variance inflation.

11.3 Masked Cross-Entropy Loss

$$\mathcal{L}_{\text{CE}} = -\log\frac{\exp(\tilde{\ell}_{c^*})}{\sum_j \exp(\tilde{\ell}_j)},$$

where c^* is the true bin index and $\tilde{\ell}$ are masked logits.

11.4 Combined Training Loss (ProbReg)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{regression}} + \lambda_{\text{CE}} \cdot \mathcal{L}_{\text{CE}} + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{regularization}} + \lambda_{\text{boundary}} \cdot \mathcal{L}_{\text{boundary}} + \mathcal{L}_{\text{selection}}. \tag{21}$$

Currently $\lambda_{\text{CE}} = 1$ (implicit, not tuned).

11.5 Boundary Regularization Loss

$$\mathcal{L}_{\text{boundary}} = \frac{1}{N}\sum_{i=1}^{N}\left[\max(0, y_{\min} - \hat{y}_i)^2 + \max(0, \hat{y}_i - y_{\max})^2\right].$$

11.6 Middle-Class Penalty

When K is odd, penalize the middle class $k_{\text{mid}} = \lceil K/2 \rceil$:

$$\mathcal{L}_{\text{mid}} = -\log\mathcal{N}(\mu_{k_{\text{mid}}}; \bar{y}_{\text{mid}}, \sigma^2_{k_{\text{mid}}}).$$

11.7 Tree-Model NLL Objective

See §4.2 (in the GBT section) for the custom gradients and Hessians used by XGBoost, LightGBM, and CatBoost.

12 Learned Regularization

Instead of fixing λ_1, λ_2, they are learned via separate optimizer steps. Let d be the number of model parameters.

12.1 L1 Only (Laplace Prior)

$$\mathcal{L}_{\text{reg}} = -d \cdot \log \frac{\lambda_1}{2} + \lambda_1 \|\mathbf{w}\|_1, \quad \lambda_1 = e^{\ell_1}.$$

12.2 L2 Only (Gaussian Prior)

$$\mathcal{L}_{\text{reg}} = -\frac{d}{2} \log \frac{\lambda_2}{\pi} + \lambda_2 \|\mathbf{w}\|_2^2.$$

12.3 L1 + L2 (Horseshoe-like Prior)

$$\mathcal{L}_{\text{reg}} = d \cdot \log Z + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|_2^2, \quad \log Z = \frac{1}{2} \log \frac{\pi}{\lambda_2} + \frac{\lambda_1^2}{4\lambda_2} + \log \operatorname{erfc}\left(\frac{\lambda_1}{2\sqrt{\lambda_2}}\right).$$

13 Target Transforms

13.1 Symmetric Log Transform (Symlog)

For targets spanning multiple orders of magnitude:

$$\text{symlog}(x) = \text{sign}(x) \cdot \log(1 + |x|), \tag{22}$$

$$\text{symexp}(y) = \text{sign}(y) \cdot (\exp(|y|) - 1). \tag{23}$$

symexp is the exact inverse. Compresses large values ($\text{symlog}(1000) \approx 6.9$) while preserving sign.

13.2 Uncertainty Conversion: Monte Carlo

If the model predicts (μ_s, σ_s^2) in symlog space:

1. Sample $\epsilon_1, \ldots, \epsilon_M \sim \mathcal{N}(\mu_s, \sigma_s^2)$, $M = 200$.
2. Apply: $z_m = \text{symexp}(\epsilon_m)$.
3. Return: $\hat{\mu} = \frac{1}{M} \sum z_m$, $\hat{\sigma} = \sqrt{\frac{1}{M} \sum (z_m - \hat{\mu})^2}$.

Exact up to MC variance; handles nonlinearity of symexp near zero.

14 Training Loop

All neural models follow this loop (`base_pytorch.py`):

Algorithm 2 Training loop for PyTorch models

Require: Model $f_{\boldsymbol{\theta}}$, data $(\mathbf{X}, \mathbf{y})$, learning rate η, patience P

```
Split into train/validation
Initialize Adam optimizer
best_loss ← ∞, wait ← 0
for epoch = 1 to max_epochs do
    for each mini-batch (X_b, y_b) do
        ŷ_b ← f_θ(X_b)
        L ← criterion(ŷ_b, y_b) + L_reg(θ)
        Backprop and update θ via Adam
        if learned regularization then update λ params separately
        end if
    end for
    L_val ← evaluate on validation set
    if L_val < best_loss then
        Save checkpoint, wait ← 0
    else
        wait ← wait + 1
        if wait ≥ P then break
        end if
    end if
end for
Load best checkpoint
```

Part V

Proposed Baseline Models

15 NGBoost (Duan et al., 2020)

Natural gradient boosting. For $\mathcal{N}(\mu, \sigma^2)$, update at each step uses the *natural gradient*:

$$\tilde{\nabla}_{\boldsymbol{\theta}\mathcal{L}=\mathcal{I}(\boldsymbol{\theta})^{-1}\nabla}$$
$$_{\boldsymbol{\theta}\mathcal{L},} \qquad _{\mathcal{I}=}\begin{pmatrix} 1/\sigma^2 & 0 \\ 0 & 1/(2\sigma^4) \end{pmatrix}.$$

This accounts for the geometry of the parameter space, preventing over-aggressive σ updates.

Uncertainty: Learned $\hat{\sigma}^2(\mathbf{x})$, input-dependent via tree structure. Primary tree-based competitor to ProbReg.

16 Mixture Density Network — MDN (Bishop, 1994)

A neural network outputting M-component Gaussian mixture parameters:

$$p(y \mid \mathbf{x}) = \sum_{m=1}^{M} w_m(\mathbf{x}) \cdot \mathcal{N}(y; \mu_m(\mathbf{x}), \sigma_m^2(\mathbf{x})), \tag{24}$$

with $w_m = \mathrm{softmax}(\ell^w)_m$, $\sigma_m = \exp(\ell_m^\sigma/2)$.

Loss: $\mathcal{L}_{\mathrm{MDN}} = -\frac{1}{N}\sum_i \log\left(\sum_m w_m(\mathbf{x}_i) \cdot \phi(y_i; \mu_m, \sigma_m^2)\right)$.

Uncertainty: Full mixture variance via law of total variance: $\hat{\sigma}^2 = \sum_m w_m \sigma_m^2 + \sum_m w_m(\mu_m - \hat{\mu})^2$.

vs ProbReg: MDN has no structural constraint on components; ProbReg imposes a classification bottleneck that regularizes the mixture.

17 Deep Ensembles (Lakshminarayanan et al., 2017)

Train M independently initialized networks, each outputting (μ_m, σ_m^2):

$$\hat{\mu} = \frac{1}{M}\sum_{m=1}^{M} \mu_m, \qquad \hat{\sigma}^2 = \frac{1}{M}\sum_{m=1}^{M} \sigma_m^2 + \frac{1}{M}\sum_{m=1}^{M} (\mu_m - \hat{\mu})^2. \tag{25}$$

Same law-of-total-variance decomposition as ProbReg with $\pi_m = 1/M$.

vs ProbReg: M full-size networks ($O(M)$ forward passes) vs lightweight per-class heads behind a shared classifier ($O(1)$ forward passes).

18 Quantile Regression NN (Koenker, 2005)

Trained with pinball loss at multiple quantiles $\tau_1 < \cdots < \tau_Q$:

$$\mathcal{L}_{\mathrm{QR}} = \frac{1}{NQ}\sum_{i,j} L_{\tau_j}(y_i, \hat{q}_{\tau_j}(\mathbf{x}_i)), \quad L_\tau(y, \hat{q}) = \begin{cases} \tau(y - \hat{q}) & y \geq \hat{q}, \\ (1-\tau)(\hat{q} - y) & y < \hat{q}. \end{cases}$$

Distribution-free. Intervals: $[\hat{q}_{\alpha/2}, \hat{q}_{1-\alpha/2}]$.

19 Gaussian Process

$$\hat{\mu}(\mathbf{x}^*) = \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \tag{26}$$

$$\hat{\sigma}^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*. \tag{27}$$

Training $O(N^3)$; limited to $N \leq 5000$.

20 Constant-Variance NN

$f_{\boldsymbol{\theta}(\mathbf{x})=(\mu(\mathbf{x}),\log\sigma^2)}$ where $\log \sigma^2$ is a single global parameter.

Implementation Note

Implemented as ProbReg with $K = 1$, `uncertainty_method = PROBABILISTIC`.

21 FT-Transformer (Gorishniy et al., 2021)

Tabular transformer: tokenizes each feature into an embedding and applies multi-head self-attention.

Feature tokenization: For feature j: $\mathbf{e}_j = \mathbf{w}_j \cdot x_j + \mathbf{b}_j \in \mathbb{R}^{d_{\text{emb}}}$. Prepend `[CLS]` token $\mathbf{e}_0$.

Transformer blocks (L layers):

$$\tilde{\mathbf{E}}^{(l)} = \mathbf{E}^{(l-1)} + \text{MultiHeadAttn}(\text{LN}(\mathbf{E}^{(l-1)})), \tag{28}$$

$$\mathbf{E}^{(l)} = \tilde{\mathbf{E}}^{(l)} + \text{FFN}(\text{LN}(\tilde{\mathbf{E}}^{(l)})). \tag{29}$$

Prediction: $\hat{y} = \mathbf{w}_{\text{out}}^\top \mathbf{e}_0^{(L)} + b_{\text{out}}$.

Uncertainty: No native UQ. Can combine with Deep Ensembles, MC Dropout, or conformal prediction.

Proposed extension (Paper C): Apply FlexibleNN depth gating to FT-Transformer blocks — variable self-attention depth per input with ELBO regularization.

22 TabPFN (Hollmann et al., 2024)

Pre-trained transformer performing in-context learning:

$$\hat{y}(\mathbf{x}^*) = \text{TabPFN}(\mathbf{x}^*, \mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}).$$

No task-specific training. Pre-trained on millions of synthetic datasets.

Uncertainty: Outputs a predictive distribution (quantile predictions for intervals).

Limits: $N \leq 10{,}000$, $d \leq 500$.

23 Laplace-Approximation BNN (Daxberger et al., 2021)

Post-hoc Bayesian: train MAP estimate $\boldsymbol{\theta}_{\text{MAP}}$, approximate posterior as Gaussian: $p(\boldsymbol{\theta} \mid \mathcal{D}) \approx \mathcal{N}(\boldsymbol{\theta}_{\text{MAP}}, (\nabla^2 \mathcal{L})^{-1})$.

Predictive variance via linearization:

$$\hat{\sigma}^2(\mathbf{x}^*) = \sigma_n^2 + \mathbf{J}(\mathbf{x}^*)^\top (\nabla^2 \mathcal{L})^{-1} \mathbf{J}(\mathbf{x}^*), \tag{30}$$

where $\mathbf{J}(\mathbf{x}^*) = \nabla_{\boldsymbol{\theta} f(\mathbf{x}^*; \boldsymbol{\theta}_{\text{MAP}})}$.

Available via `laplace-torch` (∼50 lines wrapper).

24 Conformal Prediction

Distribution-free intervals that wrap *any* fitted model.

Given calibration set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and miscoverage α:

1. Nonconformity scores: $R_i = |y_i - \hat{f}(\mathbf{x}_i)|$.
2. Finite-sample-corrected quantile:

$$\hat{q}_\alpha = \text{Quantile}\left(\{R_i\},\ \frac{\lceil (n+1)(1-\alpha) \rceil}{n}\right). \tag{31}$$

3. Prediction interval: $C(\mathbf{x}) = [\hat{f}(\mathbf{x}) - \hat{q}_\alpha,\ \hat{f}(\mathbf{x}) + \hat{q}_\alpha]$.

Definition 24.1 (Coverage Guarantee)**.** Under exchangeability: $\Pr(y \in C(\mathbf{x})) \geq 1 - \alpha$.

Remark 24.1. Marginal coverage (constant width). Locally-adaptive conformal (normalizing by $\hat{\sigma}(\mathbf{x})$) is a planned extension.

Part VI

Evaluation

25 Evaluation Metrics

25.1 Currently Implemented

$$\text{MAE} = \frac{1}{N}\sum_i |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2}, \tag{32}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad \text{MAPE} = \frac{100}{N}\sum_i \frac{|y_i - \hat{y}_i|}{|y_i|}. \tag{33}$$

NLL: $\text{NLL} = \frac{1}{N}\sum_i \frac{1}{2}\left(\log(2\pi) + \log\hat{\sigma}_i^2 + (y_i - \hat{y}_i)^2/\hat{\sigma}_i^2\right)$.
PICP: $\text{PICP}_\alpha = \frac{1}{N}\sum_i \mathbf{1}[y_i \in [\hat{y}_i - z_{\alpha/2}\hat{\sigma}_i, \hat{y}_i + z_{\alpha/2}\hat{\sigma}_i]]$.
ECE (current): Non-standard classification-style binning (bug N3).

25.2 Proposed Additions

25.2.1 CRPS

For $\mathcal{N}(\mu, \sigma^2)$:

$$\text{CRPS} = \sigma\left[\frac{y-\mu}{\sigma}\left(2\Phi\left(\frac{y-\mu}{\sigma}\right) - 1\right) + 2\phi\left(\frac{y-\mu}{\sigma}\right) - \frac{1}{\sqrt{\pi}}\right]. \tag{34}$$

Strictly proper scoring rule.

25.2.2 Winkler Score

For interval $[L, U]$ at level $(1-\alpha)$:

$$W_\alpha = (U - L) + \frac{2}{\alpha}(L - y)\mathbf{1}[y < L] + \frac{2}{\alpha}(y - U)\mathbf{1}[y > U]. \tag{35}$$

25.2.3 PIT-Based Calibration (Kuleshov et al., 2018)

$U = F(Y \mid \mathbf{x}) \sim \text{Uniform}(0, 1)$ when calibrated.
Calibration curve: $\hat{p}_j = \frac{1}{N}\sum_i \mathbf{1}[F(y_i|\mathbf{x}_i) \le p_j]$.
Regression ECE: $\text{ECE} = \frac{1}{B}\sum_j |p_j - \hat{p}_j|$.
Miscalibration area: $\int_0^1 |\hat{p}(t) - t|\, dt$.

25.2.4 Sharpness

Sharpness $= \frac{1}{N}\sum_i \hat{\sigma}_i$.

25.2.5 Pinball Loss

$L_\tau(y, \hat{q}_\tau) = \max(\tau(y - \hat{q}_\tau), (1-\tau)(\hat{q}_\tau - y))$. At $\tau = 0.5$: MAE.

25.2.6 MPIW

$\mathrm{MPIW}_\alpha = \frac{1}{N}\sum_i (U_i - L_i)$.

26 Photometric Redshift Metrics

Normalized residual: $\Delta z_i = (z_{\mathrm{phot},i} - z_{\mathrm{spec},i})/(1 + z_{\mathrm{spec},i})$.

Bias: $\langle \Delta z \rangle$. **Scatter:** $\sigma_{\mathrm{MAD}} = 1.4826 \cdot \mathrm{median}(|\Delta z_i - \mathrm{median}(\Delta z)|)$. **Outlier fraction:** $\eta = \frac{1}{N}\sum_i \mathbf{1}[|\Delta z_i| > 0.15]$.

PIT uniformity: KS test on $\{F(z_{\mathrm{spec},i}|\mathbf{x}_i)\}$ vs $\mathrm{Uniform}(0,1)$.

A Known Bugs Affecting the Mathematics

ID	Severity	Mathematical Impact
N1	MEDIUM	FlexibleNN ELBO depth prior uses `arange` instead of `linspace(3,1)`, causing prior steepness to scale with $D_{\max}$.
N2	HIGH	STE gradient path broken — indexed assignment instead of weighted sum. No gradient flows through gating parameters.
N3	LOW	ECE uses classification-style binning instead of PIT-based formulation. Reported values are non-standard.
N4	LOW	β-NLL omits $\frac{1}{2}\log(2\pi)$. Gradients and rankings unaffected.
N5	MEDIUM	Tree Hessian for $\log \sigma^2$ uses observed Hessian instead of Fisher ($\frac{1}{2}$). Can cause negative Hessians.
N6	LOW	`build_model` called twice. No math impact.
N7	MEDIUM	NoneStrategy (layer selection) wrong-size probability vector. Latent crash.